

# Audio Compression Project

## CODE SUBMISSION

This project implements a full pipeline:

Input Audio → Separation → Compression → Reconstruction → Evaluation

### 1.Setup

Step 1: Install tools

```
sudo apt update
```

```
sudo apt install -y audacity ffmpeg python3 python3-pip
```

```
pip3 install numpy librosa matplotlib soundfile pydub
```

```
pip install demucs
```

```
pip install pydub
```

```
pip install mutagen
```

Step 2: Enable Audacity scripting

Edit → Preferences → Modules → mod-script-pipe → Enabled

Step 3: Check FFmpeg installation and audio codecs

```
ffmpeg -version
```

```
ffmpeg -encoders | grep aac
```

Step 4: Prepare project folders

```
mkdir -p audio-project/output/stems
```

```
mkdir -p audio-project/output/compressed
```

```
mkdir -p audio-project/output/reconstructed
```

### 2.Python program

#### 2.1.Audio separation and compression

Save this as `analyze_single.py`

```
import librosa
```

```
import librosa.display
```

```
import matplotlib.pyplot as plt
```

```

import numpy as np

import os

# Danh sách các file cần phân tích

stems = ['vocalcompress.wav', 'drumcompress.wav', 'basscompress.wav',
'othercompress.wav']

current_dir = os.path.dirname(os.path.abspath(__file__))

# Tạo khung hình lớn (4 hàng tương ứng 4 loại nhạc cụ)

fig, axes = plt.subplots(nrows=len(stems), ncols=2, figsize=(18, 15))

for i, stem_name in enumerate(stems):

    path = os.path.join(current_dir, stem_name)

    if not os.path.exists(path):

        print(f'Bỏ qua: Không tìm thấy {stem_name}')

        continue

    # Load file

    y, sr = librosa.load(path, sr=None)

    # 1. Vẽ Waveform + RMS Energy (Cột bên trái)

    rms = librosa.feature.rms(y=y)[0]

    times = librosa.times_like(rms)

    axes[i, 0].plot(np.linspace(0, len(y)/sr, len(y)), y, alpha=0.5,
label='Waveform')

    axes[i, 0].plot(times, rms, color='red', linewidth=2, label='RMS Energy
(Compression Level)')

    axes[i, 0].set_title(f'Dynamics Analysis: {stem_name}')

    axes[i, 0].legend(loc='upper right')


    # 2. Vẽ Spectrogram (Cột bên phải)

    S = librosa.feature.melspectrogram(y=y, sr=sr)

    S_dB = librosa.power_to_db(S, ref=np.max)

```

```
img = librosa.display.specshow(S_dB, sr=sr, x_axis='time', y_axis='mel',  
ax=axes[i, 1])
```

```
axes[i, 1].set_title(f'Frequency Content: {stem_name}')
```

```
fig.colorbar(img, ax=axes[i, 1], format='%+2.0f dB')
```

```
plt.tight_layout()
```

```
plt.show()
```

Run:

```
python3 analyze_single.py vocalcompress.wav
```

```
python3 analyze_single.py drumscompress.wav
```

```
python3 analyze_single.py basscompress.wav
```

```
python3 analyze_single.py othercompress.wav
```

## 2.2.Reconstruct

Save this as reconstruct.py

```
from pydub import AudioSegment
```

```
import os
```

```
# Danh sách chính xác các file stems đã nén của bạn
```

```
stems = ['vocalscompress.wav', 'drumscompress.wav', 'basscompress.wav',  
'othercompress.wav']
```

```
combined = None
```

```
print("--- Đang bắt đầu quá trình Reconstruct Mix ---")
```

```
for stem in stems:
```

```
    if os.path.exists(stem):
```

```
        print(f'Đang trộn: {stem}')
```

```
        audio = AudioSegment.from_wav(stem)
```

```
        if combined is None:
```

```
            combined = audio
```

```
        else:
```

```
            # Chồng các track lên nhau
```

```

        combined = combined.overlay(audio)

    else:

        print(f'Cảnh báo: Không tìm thấy file {stem}')

if combined:

    combined.export("reconstructed_mix.wav", format="wav")

    print("\n--- THÀNH CÔNG! ---")

    print("File hỗn hợp đã được tạo: reconstructed_mix.wav")

else:

    print("Thất bại: Không có file nào để gộp.")

```

Run:

```
python3 reconstruct.py
```

### 2.3.Showing bitrate

Save this as show\_bitrate.py

```

import subprocess
import os

# Danh sách các file stems của bạn
stems = ['vocalscompress.wav', 'drumscompress.wav', 'basscompress.wav',
'othercompress.wav']

print(f'{'File Name':<20} | {'Bitrate':<15} | {'Sample Rate':<15}')
print("-" * 55)

for stem in stems:
    if os.path.exists(stem):
        # Sử dụng ffprobe để trích xuất thông tin
        cmd = f'ffprobe -v error -select_streams a:0 -show_entries
stream=bit_rate,sample_rate -of default=noprint_wrappers=1:nokey=1 {stem}'
        result = subprocess.check_output(cmd, shell=True).decode().split()

        # Nếu là file WAV, bit_rate thường trả về N/A, ta tính toán thủ công:
        # Bitrate (bps) = SampleRate * 16 (bit) * 2 (channels)
        sample_rate = result[0]
        bitrate_kbps = (int(sample_rate) * 16 * 2) // 1000

```

```
print(f'{stem:<20} | {bitrate_kbps:>6} kbps | {sample_rate:>10} Hz")
else:
    print(f'{stem:<20} | File Not Found")
```

Run:

```
python3 show_bitrate.py
```

### 3.Setup instructions

#### Overview

This project implements a complete audio processing pipeline:

Input Audio → Separation → Compression → Reconstruction → Evaluation

The system combines:

- **Audacity** for audio preprocessing and separation
- **Python** for compression, reconstruction, and analysis
- **FFmpeg** for audio encoding and bitrate inspection

#### Setup

##### 1. Install required tools

```
sudo apt update
```

```
sudo apt install -y audacity ffmpeg python3 python3-pip
```

##### 2. Install Python libraries

```
pip3 install numpy librosa matplotlib soundfile pydub mutagen demucs
```

##### 3. Enable Audacity scripting (optional)

Open Audacity:

Edit → Preferences → Modules → mod-script-pipe → Enabled

Restart Audacity after enabling.

##### 4. Check FFmpeg

```
ffmpeg -version
```

```
ffmpeg -encoders | grep aac
```

##### 5. Create project folders

```
mkdir -p output/stems
```

```
mkdir -p output/compressed
```

```
mkdir -p output/reconstructed
```

## **Audio Processing Workflow**

### **Step 1: Source Separation**

- Import audio into Audacity
- Separate into stems (vocals, drums, bass, others)
- Export as:

```
output/stems/
```

```
|— vocals.wav
```

```
|— drums.wav
```

```
|— bass.wav
```

```
|— other.wav
```

### **Step 2: Compression**

- Apply compression (quantization or FFmpeg encoding)
- Output files:

```
output/compressed/
```

```
|— vocalscompress.wav
```

```
|— drumscompress.wav
```

```
|— basscompress.wav
```

```
|— othercompress.wav
```

### **Step 3: Reconstruction**

Run:

```
python3 reconstruct.py
```

Output:

```
output/reconstructed/reconstructed_mix.wav
```

### **Step 4: Bitrate Analysis**

Run:

```
python3 show_bitrate.py
```

Displays:

- Bitrate (kbps)
- Sample rate (Hz)

### **Step 5: Audio Analysis (Waveform + Spectrogram)**

Run:

```
python3 analyze_single.py
```

This will display:

- Waveform
- RMS energy (compression level)
- Spectrogram

### **Project Structure**

project/

```
|— output/
|  |— stems/
|  |— compressed/
|  |— reconstructed/
|
|— analyze_single.py
|— reconstruct.py
|— show_bitrate.py
|
|— requirements.txt
|— README.md
```

### **Output**

- Reconstructed audio file
- Bitrate comparison

- Waveform and spectrogram visualization

## Notes

- Audacity is used for manual preprocessing and listening
- Python handles compression and reconstruction
- FFmpeg is used for bitrate analysis

## Run Summary

python3 analyze\_single.py

python3 reconstruct.py

python3 show\_bitrate.py







